



Stock Portfolio Management System

Welcome to our web-based stock portfolio management system built with Python and Flask. This application allows users to efficiently track, analyze, and manage their stock investments through an intuitive interface.

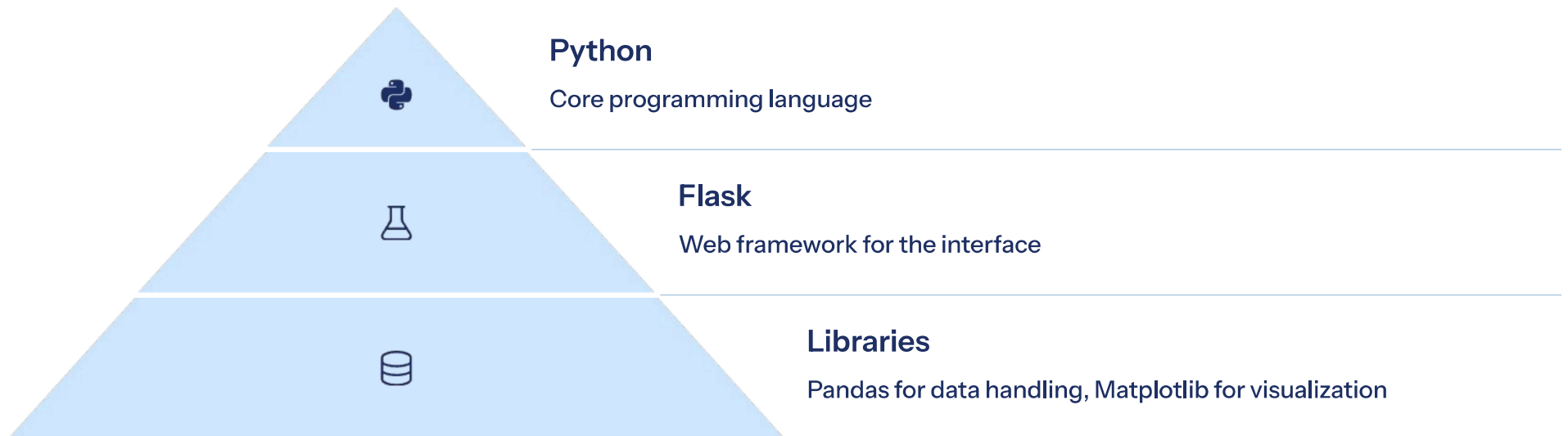
Our system combines powerful data structures with visualization capabilities to provide a complete solution for investors. Throughout this presentation, we'll explore the key features, technical implementation, and practical benefits of this portfolio management tool.

By Prem Panchal Reg No: **2411021240011**

By Adarsh Gupta Reg No: **2411021240076**

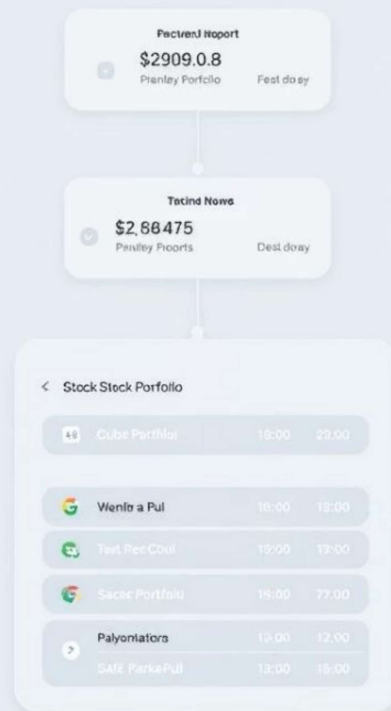
By Suraj Mal Reg No: **2411021240000**

Technology Stack Overview



Our application is built on a robust technology stack centered around Python, providing flexibility and powerful data processing capabilities. Flask serves as the lightweight web framework that delivers the interface via HTTP server, making the application accessible through any web browser.

We leverage Pandas for efficient data manipulation and Matplotlib for creating insightful visualizations of portfolio performance. The entire development environment is integrated within Jupyter Notebook for streamlined coding and testing.



Data Structure Implementation

Portfolio Storage

Python list of dictionaries provides flexible and efficient data management for stock records

Stock Dictionary

Each stock entry contains ticker symbol, buy price, quantity, and current price

Dynamic Updates

List structure allows for easy addition, removal, and modification of stock entries

The foundation of our portfolio management system is a straightforward yet effective data structure. We utilize a Python list to store dictionaries, with each dictionary representing an individual stock holding. This approach provides the flexibility needed for dynamic portfolio management while maintaining simplicity in the codebase. The structure enables efficient operations like filtering, sorting, and calculating aggregate values across the entire portfolio, which are essential for providing meaningful insights to users.

Remove

Ticker to Update

New Quantity

New Price

Update

Search Ticker

Flask Routes Architecture



The application's functionality is organized through a series of Flask routes that handle different aspects of portfolio management. Each route serves a specific purpose, from basic CRUD operations to advanced analysis and visualization features.

Core Portfolio Management Features



Add Stocks

Easily add new stocks with ticker symbol, purchase price, quantity, and current market price



Update Holdings

Modify quantity or current price of existing stocks to reflect market changes or additional purchases



Remove Stocks

Delete stocks from the portfolio when positions are closed or no longer relevant



Search Functionality

Quickly find specific stocks by ticker symbol to view detailed information

The system provides a comprehensive set of features for managing portfolio entries. Users can perform all essential operations through a clean and intuitive interface, with changes reflected immediately in the portfolio view.

Portfolio Analysis Capabilities

Summary View

The summary route calculates and displays key portfolio metrics:

- Total invested amount
- Current portfolio value
- Overall profit or loss

This provides users with a quick snapshot of their portfolio performance at a glance.

Summary

Total Invested: ₹50000.0

Current Value: ₹55000.0

Profit/Loss: ₹5000.0

[Back](#)

Performance Sorting

The sort route arranges stocks by profit percentage in descending order, helping users identify:

- Best performing investments
- Underperforming positions
- Profit amount for each holding

This feature enables data-driven decision making for portfolio optimization.

Visualization with Matplotlib

Data Preparation

Extract ticker symbols as labels and calculate current value (quantity × price) for each holding to use as data points

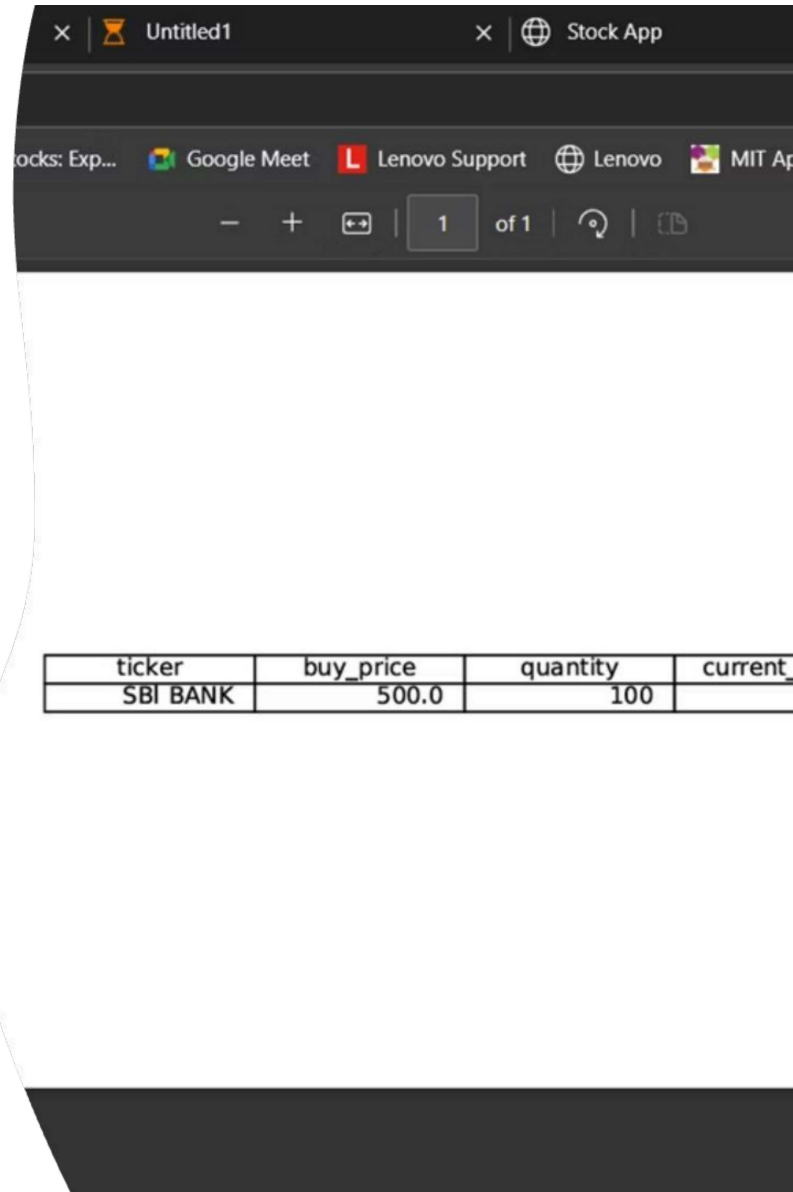
Chart Generation

Create a pie chart using matplotlib that visually represents the proportion of each stock in the overall portfolio

Web Integration

Save the chart as an image in the static folder and display it in the browser through the '/chart' route

The visualization component transforms raw portfolio data into an intuitive pie chart that helps users understand their asset allocation at a glance. This graphical representation makes it easy to identify overexposure to particular stocks or sectors, supporting better diversification decisions.



[illegible]

Export Functionality



CSV Export

Converts portfolio data to a structured CSV file using `pandas.DataFrame.to_csv()`, allowing for easy import into spreadsheet applications for further

analysis



PDF Export

Generates a professional PDF document containing the portfolio table using `matplotlib.table` inside a figure, providing a shareable and printable



Chart View

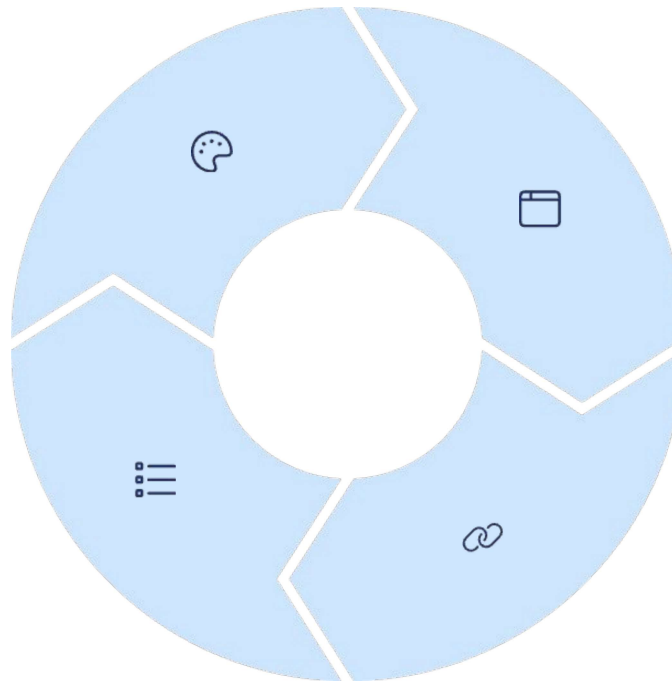
Creates and displays a visual representation of the portfolio allocation, offering insights into diversification and asset distribution

The export features enhance the utility of the application by enabling users to work with their portfolio data outside the web interface. Whether for record-keeping, sharing with financial advisors, or conducting more detailed analysis in specialized tools, these export options provide flexibility in how users leverage their investment data.

Interface Design

Clean Layout
Simple and intuitive HTML interface rendered
via Flask's template engine

Portfolio Display
Clear listing of all stocks with key information
visible at a glance



Responsive Forms
Dedicated input forms for each portfolio
management operation

Navigation Links
Quick access to summary, sorting, chart, and
export features

The user interface is designed with simplicity and functionality in mind. By using inline HTML templates, the application maintains a lightweight footprint while providing all necessary controls for effective portfolio management. The clean design ensures that users can focus on their investment data without distractions.

All operations are handled through Flask routes and reflected immediately in the UI, creating a responsive experience that feels like a desktop application despite running in a web browser.

Project Outcomes & Applications

4

Key Components

Data structures, web development, visualization, and file handling integrated into one system

9+

Flask Routes

Comprehensive set of endpoints handling everything from basic CRUD operations to advanced analysis

3

Export Options

Multiple ways to extract and utilize portfolio data outside the application

This project successfully demonstrates the practical application of fundamental programming concepts to create a useful financial tool. The stock portfolio management system showcases how Python's ecosystem of libraries can be leveraged to build a complete web application with sophisticated features.

The system is ideal for individual investors looking to track their investments without complex financial software. It could be extended with features like API integration for real-time pricing, historical performance tracking, or investment strategy testing.